RGB-D 书本厚度与夹爪开口计算——客户操作手册

目录

1. 设置路径 1
2. 检查扫描数据 2
3. 检查 Python 和 GPU 2
4. 一次性安装 Record3D 解码依赖 3
5. 检查 SAM3 模型 3
6. 使用默认参数运行 3
    6.1 只用单帧 RGB-D 运行 3
7. 查看边界与书本编号 8
8. 查询指定书本的夹爪开口 9
9. 在机器人程序中读取 10
10. 修改安全间隙 10
11. 增加多视角帧数 11
12. 指定其他 GPU 11
13. 指定自定义输出目录 11
14. 输出文件说明 11
15. 结果质量检查 12
16. 常见问题 13
    提示 `No module named 'liblzfse'` 13
    提示 RGB 与 depth 宽高比不一致 13
    厚度整体大约相差 1000 倍 13
    提示找不到 SAM3 权重 13
    GPU 显存不足 13
    检测书本数量不正确 14
17. 上线前安全要求 14

需要了解算法公式、平面分解、侧面剔除和安全拒绝条件时，请查看 `BOOK_THICKNESS_ALGORITHM.md`。

本手册用于从通用 RGB-D 文件或 Record3D 扫描数据中自动识别每一本书，计算书脊厚度，并生成双指夹爪的建议预张开宽度。

系统输出使用以下公式：

```
建议夹爪开口 = 书本厚度 + 95% 测量不确定度 + 安全间隙
```

默认安全间隙为 3 mm。夹爪接触书本后，仍应使用力控或电流阈值完成闭合。

1. 设置路径

打开终端，依次执行：

```
export PROJECT=/mnt/ssd/user/V/feature-splatting-inria
export CAPTURE=$PROJECT/2026-09-01--23-22-25
export PYTHON=/mnt/ssd/robotarm/env/sam3_full/bin/python
```

命令说明：

- `PROJECT`：厚度计算程序所在目录。
- `CAPTURE`：本次 Record3D 扫描数据目录。
- `PYTHON`：已经安装 SAM3 和 PyTorch 的 Python 解释器。

如需分析其他扫描，只需修改 `CAPTURE`：

```
export CAPTURE=/path/to/another-record3d-capture
```

2. 检查扫描数据

执行：

```
test -f "$CAPTURE/metadata" && echo "metadata: OK"
test -f "$CAPTURE/rgbd/0.jpg" && echo "RGB: OK"
test -f "$CAPTURE/rgbd/0.depth" && echo "depth: OK"
test -f "$CAPTURE/rgbd/0.conf" && echo "confidence: OK"
```

本节检查的是 Record3D 输入，多视角模式四项均应显示 `OK`。如果使用 `CAPTURE --single-frame-rgbd` 兼容模式，只要求 `metadata`、`.jpg` 和 `.depth`。如果直接使用 `--rgb`、`--depth`、`--intrinsics` 三个通用路径，则不需要 Record3D 目录和 `metadata`，请直接查看 6.1 节。Record3D 数据目录的基本结构为：

```
CAPTURE/
├── metadata
└── rgbd/
    ├── 0.jpg
    ├── 0.depth
    ├── 0.conf
    ├── 1.jpg
    └── ...
```

其中：

- `.jpg` 是高分辨率 RGB 图像。
- `.depth` 是 Record3D LiDAR 深度。
- `.conf` 是深度置信度；单帧 RGB-D 模式可省略。
- `metadata` 必须包含相机内参和图像尺寸；只有多视角模式要求其中包含位姿。

3. 检查 Python 和 GPU

检查 Python：

```
"$PYTHON" --version
```

检查 PyTorch 与 GPU：

```
"$PYTHON" -c "import torch; print('PyTorch:', torch.__version__); print('CUDA:', torch.cuda.is_available
```

正常情况下应看到：

```
CUDA: True
GPU: NVIDIA ...
```

如果显示 `CUDA: False`，请先检查 NVIDIA 驱动：

```
nvidia-smi
```

4. 一次性安装 Record3D 解码依赖

首次部署时执行：

```
"$PYTHON" -m pip install pyliblzfse
```

验证安装：

```
"$PYTHON" -c "import liblzfse; print('Record3D decoder: OK')"
```

看到 `Record3D decoder: OK` 即表示安装成功。此步骤只需执行一次。

5. 检查 SAM3 模型

当前部署使用以下模型：

```
export SAM3_CHECKPOINT=/home/vlmdemo/.cache/huggingface/hub/models--facebook--sam3/snapshots/3c879f3982
test -f "$SAM3_CHECKPOINT" && echo "SAM3 checkpoint: OK"
```

如果没有显示 `OK`，需要将 SAM3 权重放到该位置，或在运行时通过 `--checkpoint` 指定实际路径。

6. 使用默认参数运行

进入项目目录：

```
cd "$PROJECT"
```

执行完整分析：

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE"
```

逐行说明：

- `CUDA_VISIBLE_DEVICES=0`：使用第 1 块 GPU。
- `"$PYTHON"`：使用 SAM3 专用 Python 环境。
- `book_thickness_pipeline.py`：运行厚度计算程序。
- `"$CAPTURE"`：指定要分析的 RGB-D 扫描。

默认行为：

- 使用帧 `0,12,24,36,48,60,71`。
- 使用第 24 帧作为边界和编号参考图。
- 按每层书架从左到右编号。
- 过滤低置信度和重复分割。
- 融合多个视角的厚度测量。
- 在 95% 不确定度之外再增加 3 mm 夹爪间隙。

正常结束时会显示类似：

```
Measured 96 books
Per shelf: S1=29, S2=33, S3=34
Quality: {'high': 91, 'medium': 5, 'low': 0}
Results: .../book_thickness_analysis
```

6.1 只用单帧 RGB-D 运行

6.1.1 什么时候使用 如果只有一张 RGB 图、一张已配准深度图和一个相机内参文件，可以直接输入这三个 文件的路径。数据不必来自 Record3D，也不要求固定的目录结构。

同时提供 `--rgb`、`--depth` 和 `--intrinsics` 后，程序会自动进入通用单帧模式，不需要提供 `CAPTURE`、`metadata`、`.conf`、位姿或帧编号。

该模式的行为是：

- 直接读取命令行指定的 RGB、depth 和内参文件。
- 不依赖 Record3D 的 `metadata` 和 `rgbd/` 目录。
- 不读取深度 confidence，不使用相机位姿。
- 不进行多视角匹配和厚度融合。
- 仍使用相机内参将图像像素宽度换算为毫米。

原来的 Record3D 单帧命令仍然兼容，详见 6.1.5。

6.1.2 三个输入文件 三个文件可以放在任意目录，例如：

```
/data/books/rgb.png
/data/books/depth.png
/data/books/intrinsics.json
```

RGB 支持常用的 JPEG、PNG 和 TIFF。

depth 支持：

- 单通道 PNG 或 TIFF。
- `.npy` 二维 NumPy 数组。
- `.npz` 文件；数组键建议命名为 `depth`。如果只有一个数组也会自动读取。

深度单位规则：

- 浮点深度默认按米读取。
- 16 位及以上整数深度默认按毫米读取。
- 如果传感器使用其他单位，必须通过 `--depth-scale` 指定每个数值对应多少米。

例如，深度图每个单位代表 0.1 mm：

```
--depth-scale 0.0001
```

推荐将相机内参保存为 JSON：

```
{
  "fx": 1348.5952,
  "fy": 1348.5952,
  "cx": 720.9918,
  "cy": 962.8973
}
```

也可以使用标准的 3×3 相机矩阵：

```
{
  "K": [
    [1348.5952, 0.0, 720.9918],
    [0.0, 1348.5952, 962.8973],
    [0.0, 0.0, 1.0]
  ]
}
```

内参文件还支持 TXT、NPY、NPZ 和 OpenCV YAML/XML。TXT 可以写 `fx fy cx cy` 四个数，也可以写按行排列的 3×3 矩阵。

内参必须对应 RGB 图像坐标系。不能直接使用未经配准的原始深度相机内参。

RGB 和 depth 必须来自同一次采集并完成空间配准。两者允许分辨率不同，但宽高比 必须接近；程序会拒绝宽高比相差超过 2% 的输入。

6.1.3 设置输入路径　执行：

```
export RGB=/data/books/rgb.png
export DEPTH=/data/books/depth.png
export INTRINSICS=/data/books/intrinsics.json
export RESULT=/data/books/book_thickness_result
```

检查文件：

```
test -f "$RGB" && echo "RGB: OK"
test -f "$DEPTH" && echo "depth: OK"
test -f "$INTRINSICS" && echo "intrinsics: OK"
```

三项都应显示 OK。

检查 RGB、depth 的尺寸、数据类型和内参：

```
RGB="$RGB" DEPTH="$DEPTH" INTRINSICS="$INTRINSICS" "$PYTHON" - <<'PY'
import json
import os
from pathlib import Path

import cv2
from PIL import Image

rgb = Image.open(os.environ["RGB"])
depth = cv2.imread(os.environ["DEPTH"], cv2.IMREAD_UNCHANGED)
intrinsics = json.loads(Path(os.environ["INTRINSICS"]).read_text())

print("RGB 尺寸:", rgb.size)
print("depth 尺寸:", None if depth is None else depth.shape)
print("depth 类型:", None if depth is None else depth.dtype)
print(" 内参:", intrinsics)
PY
```

如果 depth 是 .npy 或 .npz，上面的 OpenCV 检查不适用，可以执行：

```
DEPTH="$DEPTH" "$PYTHON" - <<'PY'
import os
import numpy as np

data = np.load(os.environ["DEPTH"], allow_pickle=False)
if isinstance(data, np.lib.npyio.NpzFile):
    print({name: data[name].shape for name in data.files})
    data.close()
else:
    print("depth 尺寸:", data.shape, " 类型:", data.dtype)
PY
```

6.1.4 执行通用单帧分析　进入项目目录：

```
cd "$PROJECT"
```

执行：

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py \
  --rgb "$RGB" \
  --depth "$DEPTH" \
  --intrinsics "$INTRINSICS" \
  --output "$RESULT"
```

逐项说明：

- `--rgb`：RGB 图像路径。
- `--depth`：已配准的单通道深度路径。
- `--intrinsics`：RGB 相机内参路径。
- `--output`：结果目录；可以省略。

如果省略 `--output`，程序默认在 RGB 所在目录创建 `RGB` 文件名 `_book_thickness/`。

如果整数深度不是毫米，增加正确的换算比例。例如每单位为 0.1 mm：

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py \
  --rgb "$RGB" \
  --depth "$DEPTH" \
  --intrinsics "$INTRINSICS" \
  --depth-scale 0.0001 \
  --output "$RESULT"
```

正常结束时会显示类似：

```
Processing frame 0...
  accepted 96 books [29, 33, 34]
Measured 96 books
Per shelf: S1=29, S2=33, S3=34
Quality: {'high': 0, 'medium': 0, 'low': 96}
Results: /data/books/book_thickness_result
```

6.1.5 Record3D 单帧兼容命令

已有 Record3D 目录时，原来的命令仍然可用：

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE" \
  --single-frame-rgbd \
  --reference-frame 24 \
  --output "$CAPTURE/book_thickness_single_frame"
```

此兼容模式从 `CAPTURE/rgbd/24.jpg` 和 `24.depth` 读取数据，并从 `metadata` 读取尺寸和内参，但不读取 `24.conf`，也不使用 `metadata.poses`。

6.1.6 程序如何计算厚度

单帧模式按以下顺序处理：

1. 在高分辨率 RGB 图像中使用 SAM3 分割每个书脊。
2. 去除重复分割，并按书架层和左右顺序分配书本 ID。
3. 根据书脊轮廓拟合长轴和横向轴，得到左右边界；倾斜书本不会直接使用水平框宽。
4. 将书脊区域映射到低分辨率深度图。
5. 不使用 `.conf`，但会删除无效、过近、过远和明显偏离中位数的深度。
6. 使用相机内参把深度像素反投影成相机坐标系中的 3D 点。

7. 每本书使用 RANSAC 分解书脊平面和可能出现的侧面平面。
8. 汇总画面内所有书本，估计共同的书脊朝向。
9. 如果掩码边缘存在与书脊方向明显不同的侧面，使用书脊平面的深度支持区裁剪侧面。
10. 将修正后的左右边界射线与书脊平面求交，计算两个 3D 交点间的毫米距离。
11. 无法区分书脊与侧面时拒绝该候选，不生成夹爪命令。
12. 根据通过检查的厚度、不确定度和安全间隙计算夹爪建议开口。

夹爪开口公式为：

```
建议开口 = 书本厚度 + 95% 工程不确定度 + 安全间隙
```

默认安全间隙是 3 mm，可以通过 `--gripper-clearance-mm` 调整。

为避免书本排列松散、斜视时把侧面算进厚度，程序会拒绝以下候选：

- 书脊平面深度内点不足。
- 书脊平面没有覆盖足够的书脊长度。
- 当前书本平面方向与整层书的共同书脊方向差别过大。
- 观察角度过于倾斜。
- 侧面占比过大。
- 已检测到侧面，但无法稳定确定书脊与侧面的交界。

被拒绝候选不会进入 `gripper_openings.json`，机器人不得为其执行自动抓取。

6.1.7 确认输入来源以及没有使用 confidence 和 pose　查看结果前设置：

通用模式已经在 6.1.3 中设置了 `RESULT`。执行：

```
RESULT="$RESULT" "$PYTHON" - <<'PY'
import json
import os
from pathlib import Path

result = Path(os.environ["RESULT"])
data = json.loads((result / "book_thickness.json").read_text())

print(" 运行模式:", data["mode"])
print(" 使用 confidence:", data["uses_confidence"])
print(" 使用相机位姿:", data["uses_camera_poses"])
print(" 处理帧:", data["frames_requested"])
print(" 输入文件:", data["input_files"])
print(" 书本数量:", data["book_count"])
print(" 拒绝数量:", data["reference_rejection_count"])
PY
```

预期结果：

```
运行模式: single_frame_rgbd
使用 confidence: False
使用相机位姿: False
处理帧: [0]
输入文件: {'rgb': '...', 'depth': '...', 'intrinsics': '...', 'depth_scale_m_per_unit': 0.001}
书本数量: < 通过几何检查的书本数 >
拒绝数量: < 未通过侧面/视角检查的候选数 >
```

通用文件没有 Record3D 帧编号，因此程序将本次观测统一记为第 0 帧。`input_files` 会记录实际输入路径以及最终采用的深度比例，便于复查。

6.1.8 如何理解单帧质量

单帧无法估计跨视角重复性，因此每本书都会输出：

```
views_observed = 1
views_used = 1
repeatability_mad_mm = 0
quality = low
```

注意：

- `repeatability_mad_mm = 0` 表示没有第二次观测，不代表测量误差为零。
- `quality = low` 是单帧模式的预期行为，不代表程序执行失败。
- `high_confidence_samples` 不代表真实的传感器高置信度点，因为没有读取 `.conf`。
- 判断是否使用 confidence，应查看顶层的 `uses_confidence`。
- 单帧 `uncertainty_95_mm` 不包含真实的跨视角波动，可能低估异常书本的误差。
- `reference_side_face_detected = true`：检测到可见侧面。
- `reference_boundary_corrected = true`：已裁剪侧面，图中书本 ID 后带 *。
- `reference_side_trim_mm`：从原轮廓宽度中去除的侧面毫米数。
- `reference_view_angle_deg`：书脊平面相对正视方向的夹角。
- `frame_statistics[].rejected_detections`：被拒绝候选及具体原因。

6.1.9 当前数据的测试结果

已使用机器人真机采集的以下三个文件完成侧面剔除测试：

```
dual_v2_1/11/rgb/000001.png
dual_v2_1/11/depth/000001.png
dual_v2_1/11/cam_K.txt
```

测试结果：

- SAM3 得到 66 个书本候选。
- 57 个候选通过书脊平面几何检查。
- 9 个候选因平面方向不一致、书脊内点不足或侧面边界不明确而被拒绝。
- 6 本书检测到侧面并完成边界裁剪。
- 共从轮廓宽度中裁掉 14.58 mm，单本裁剪 1.86–3.60 mm。
- 被拒绝候选没有写入 `gripper_openings.json`。

测试结果位于：

```
dual_v2_1/11/book_thickness_000001_plane_corrected/
```

该测试证明侧面检测和安全拒绝分支已经运行，但没有每本书的卡尺真值，因此不能仅凭 该结果宣称达到毫米级绝对精度。

6.1.10 客户使用要求

1. 相机尽量正对书架，确保书脊完整、清晰、不过曝。
2. 不要修改 RGB 或 depth 的尺寸、方向和裁剪区域。
3. 避免强反光、透明塑封和大面积无纹理表面。
4. 上线前至少抽查 5 本不同厚度的书并使用卡尺对比。
5. 单帧结果只能作为夹爪预张开初值。
6. 实际夹紧必须使用力控、力矩限制或电机电流阈值。
7. 未经卡尺标定和人工复核，不得使用单帧结果直接执行无人值守抓取。

7. 查看边界与书本编号

输出目录：

```
export RESULT=${RESULT:-$CAPTURE/book_thickness_analysis}
```

全部书本边界：

```
xdg-open "$RESULT/books_thickness_overlay.png"
```

分别查看三层书架：

```
xdg-open "$RESULT/shelf_1_thickness.png"
xdg-open "$RESULT/shelf_2_thickness.png"
xdg-open "$RESULT/shelf_3_thickness.png"
```

书本编号规则：

- S1-01：第 1 层，从左向右第 1 本。
- S2-08：第 2 层，从左向右第 8 本。
- S3-34：第 3 层，从左向右第 34 本。

查看指定书本的独立核验图：

```
xdg-open "$RESULT/book_crops/S1-05.png"
```

图中：

- 青色、橙色或绿色轮廓表示书脊边界。
- T 表示估计厚度。
- +/- 表示 95% 工程不确定度。
- OPEN 表示建议夹爪预张开宽度。

8. 查询指定书本的夹爪开口

以下命令查询 S1-05：

```
BOOK_ID=S1-05 "$PYTHON" - <<'PY'
import json
import os
from pathlib import Path

result = Path(os.environ["RESULT"])
book_id = os.environ["BOOK_ID"]
data = json.loads((result / "gripper_openings.json").read_text())
item = data["commands"][book_id]

print(" 书本 ID:", book_id)
print(" 厚度:", round(item["measured_thickness_mm"], 2), "mm")
print("95% 不确定度:", round(item["uncertainty_95_mm"], 2), "mm")
print(" 建议夹爪开口:", round(item["opening_mm"], 2), "mm")
print(" 建议夹爪开口:", round(item["opening_m"], 6), "m")
print(" 质量:", item["quality"])
PY
```

注意：RESULT 必须先导出，而不是只定义为普通 shell 变量：

```
export RESULT=$CAPTURE/book_thickness_analysis
```

查询其他书本时修改 BOOK_ID：

```
BOOK_ID=S3-12 "$PYTHON" - <<'PY'
import json
import os
from pathlib import Path

data = json.loads(
    (Path(os.environ["RESULT"]) / "gripper_openings.json").read_text()
)
item = data["commands"][os.environ["BOOK_ID"]]
print(item["opening_mm"])
PY
```

该命令只输出毫米开口值，方便传给机器人程序。

9. 在机器人程序中读取

Python 示例：

```
import json
from pathlib import Path

result = Path(
    "/mnt/ssd/user/V/feature-splatting-inria/"
    "2026-09-01--23-22-25/book_thickness_analysis"
)
book_id = "S1-05"

data = json.loads((result / "gripper_openings.json").read_text())
measurement = data["commands"][book_id]

opening_mm = measurement["opening_mm"]
opening_m = measurement["opening_m"]
quality = measurement["quality"]

if quality != "high":
    raise RuntimeError(f"{book_id} 需要人工复核")

print(f" 夹爪目标: {opening_mm:.2f} mm / {opening_m:.6f} m")
```

请根据夹爪控制接口选择 opening_mm 或 opening_m，不要混用单位。

10. 修改安全间隙

如果夹爪定位误差较大，可将安全间隙从 3 mm 增加到 5 mm：

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE" \
  --gripper-clearance-mm 5
```

该参数只影响建议开口，不改变测得的书本厚度。

建议：

- 高精度、小回差夹爪：2–3 mm。
- 一般机器人抓取：3–5 mm。

- 标定不足或运动误差较大：5 mm 以上，并配合低速接近。

11. 增加多视角帧数

需要更高的多视角稳定性时：

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE" \
  --frames 0,8,16,24,32,40,48,56,64,71 \
  --reference-frame 24
```

参数说明：

- `--frames`：参与厚度融合的帧编号。
- `--reference-frame`：用于书本编号、边界图和独立裁剪图的参考帧。

参考帧应满足：

- 三层书架尽量完整可见。
- 图像清晰，没有运动模糊。
- 相机尽量正对书脊。
- 左右边缘的书本没有被裁掉。

增加帧数通常能改善随机误差，但不能修复错误分割或严重遮挡。

12. 指定其他 GPU

使用第 2 块 GPU：

```
CUDA_VISIBLE_DEVICES=1 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE"
```

查看 GPU 编号：

```
nvidia-smi
```

13. 指定自定义输出目录

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE" \
  --output "$CAPTURE/customer_results"
```

之后设置：

```
export RESULT=$CAPTURE/customer_results
```

14. 输出文件说明

```
book_thickness_analysis/
├── book_thickness.json
├── book_thickness.csv
├── gripper_openings.json
├── books_thickness_overlay.png
├── rejected_books_overlay.png
├── shelf_1_thickness.png
├── shelf_2_thickness.png
```

```
├──── shelf_3_thickness.png
├──── reference_depth_map.png
├──── reference_book_masks.npz
└──── book_crops/
    ├──── S1-01.png
    ├──── S1-02.png
    └──── ...
```

- gripper_openings.json：机器人最方便读取的夹爪开口文件。
- book_thickness.csv：逐书汇总，可使用 Excel 打开。
- book_thickness.json：包含完整边界、逐帧测量和诊断信息。
- books_thickness_overlay.png：全部书本的边界和厚度。
- rejected_books_overlay.png：因侧面、观察角或平面质量不足而拒绝的候选。
- book_crops/：每本书的独立检查图。
- reference_book_masks.npz：每本书的高分辨率二值掩码。

15. 结果质量检查

执行：

```
RESULT="$RESULT" "$PYTHON" - <<'PY'
import json
import os
from collections import Counter
from pathlib import Path

result = Path(os.environ["RESULT"])
data = json.loads((result / "book_thickness.json").read_text())
quality = Counter(book["quality"] for book in data["books"])

print(" 书本总数:", data["book_count"])
print(" 拒绝候选:", data["reference_rejection_count"])
print(" 质量统计:", dict(quality))
print(" 需要复核:")
for book in data["books"]:
    if book["quality"] != "high":
        print(
            book["book_id"],
            " 厚度",
            round(book["thickness_mm"], 2),
            "mm, 不确定度",
            round(book["uncertainty_95_mm"], 2),
            "mm",
        )
PY
```

quality 含义：

- high：多视角一致性和匹配质量良好。
- medium：可以作为抓取初值，但建议查看独立裁剪图。
- low：不应自动执行抓取，需要重新扫描或人工确认。

常见拒绝原因：

- inconsistent_spine_plane_normal：该书平面方向与画面中多数书脊不一致。

- `low_spine_plane_inlier_ratio`：书脊平面有效深度点占比不足。
- `low_spine_plane_coverage`：有效平面没有覆盖足够的书脊长度。
- `grazing_view_angle`：观察角度过于倾斜。
- `side_face_dominates_mask`：SAM 掩码中侧面占比过高。
- `side_face_boundary_ambiguous`：检测到侧面，但无法稳定确定交界位置。
- `side_face_boundary_unresolved`：深度不足以拟合侧面裁剪边界。

出现拒绝时，应调整相机位置或增加视角，不应手工把候选重新加入夹爪命令。

16. 常见问题

提示 `No module named 'liblzfse'`

只有 Record3D `.depth` 输入需要该依赖。执行：

```
"$PYTHON" -m pip install pyliblzfse
```

通用 PNG/TIFF/NPY/NPZ 深度输入不需要 LZFSE 解码。

提示 RGB 与 depth 宽高比不一致

该错误表示 depth 没有按 RGB 图像坐标系完成配准，或者其中一个文件被单独裁剪、旋转 或缩放。应由相机 SDK 先把 depth 对齐到 RGB，不能仅通过强制缩放绕过检查。

厚度整体大约相差 1000 倍

这通常是深度单位错误：

- 浮点深度默认按米读取。
- 16 位整数深度默认按毫米读取。
- 其他单位使用 `--depth-scale` 指定 “米/数值”。

例如深度数值 10000 表示 1 m 时，比例应为：

```
--depth-scale 0.0001
```

提示找不到 SAM3 权重

运行时指定权重：

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE" \
  --checkpoint /actual/path/to/sam3.pt
```

GPU 显存不足

先查看 GPU：

```
nvidia-smi
```

选择空闲 GPU，例如：

```
CUDA_VISIBLE_DEVICES=1 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE"
```

检测书本数量不正确

先尝试更换清晰、完整的参考帧：

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE" \
  --reference-frame 12
```

仍有漏检时可略微降低阈值：

```
CUDA_VISIBLE_DEVICES=0 \
"$PYTHON" book_thickness_pipeline.py "$CAPTURE" \
  --min-score 0.45
```

降低阈值可能增加重复检测，因此每次调整后必须检查 books_thickness_overlay.png。

17. 上线前安全要求

1. 随机选择至少 5 本不同厚度的书，用卡尺测量实际厚度。
2. 对比 book_thickness.csv，确认是否存在稳定系统偏差。
3. 检查夹爪控制接口使用的单位是 mm 还是 m。
4. 首次抓取使用低速模式。
5. 夹爪闭合必须启用力控、力矩限制或电流阈值。
6. medium 和 low 质量结果不得直接进行无人值守抓取。
7. 书本紧密排列、手指无法插入时，应先执行抽书或建立间隙动作。

本系统提供抓取预张开宽度，不替代机器人碰撞检测、力控和安全控制。